

Security Review

Cantina Managed

SKY: PERMISSIONLESS SLL WITHDRAWALS

Cantina Managed review by:

Christoph Michel, Lead Security Researcher

M4rio.eth, Lead Security Researcher



August 14, 2026



Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	4
3.1	Low Risk	4
3.1.1	Untrusted callers could force ALM losses within <code>maxSharesInRatio</code> headroom . .	4
3.1.2	Protect user withdrawals against front-run penalty increases	4
3.1.3	Malicious venue can block itself from being removed via <code>setVenueType</code>	4
3.1.4	The penalty is charged when no ALM liquidity is used	5
3.2	Informational	5
3.2.1	Minor issues, Typos & Documentation	5
3.2.2	Permissionless withdrawals share rate-limit capacity with other allocators	6
3.2.3	Add a permit-based withdrawal entry point	7
3.2.4	Delisting a vault leaves its venue configuration active in storage	7
3.2.5	Use <code>AdministeredAgent</code> for <code>PermissionlessWithdrawals</code> allocator permissions	8



1 Introduction

1.1 About Cantina

Cantina is a security platform that pairs a world-class network of security researchers with agentic AI to help teams building onchain find and fix real risk. Through managed reviews, competitions, and bounties, Cantina secures protocols before vulnerabilities can be exploited.

Learn more at cantina.security

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).



2 Security Review Summary

From Jul 26th - Jul 28th the Cantina team conducted a review of [permissionless-withdrawals](#) on commit hash [87cddffe](#). The team identified a total of **9** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	4	2	2
Gas Optimizations	0	0	0
Informational	5	3	2
Total	9	5	4

The Cantina Managed team reviewed Sky's [permissionless-withdrawals](#) holistically on commit hash [d1e7ca71](#) (tag [v1.0.0](#)) and concluded that all findings were addressed and no new vulnerabilities were identified.

2.1 Scope

The security review had the following components in scope for [permissionless-withdrawals](#) on commit hash [87cddffe](#):

```
PermissionlessWithdrawals # Base contract
PermissionlessWithdrawalsDiamondPAU # Diamond interface compliant contract
PermissionlessWithdrawalsLegacyPAU # Legacy MainnetController interface compliant
↳ contract
```



3 Findings

3.1 Low Risk

3.1.1 Untrusted callers could force ALM losses within `maxSharesInRatio` headroom

Severity: Low Risk.

Context: `PermissionlessWithdrawals.sol#L368-L377`.

Description: For an ERC-4626 venue, `PermissionlessWithdrawals._withdrawFromVenue(...)` computes the maximum shares the ALM is allowed to burn from one admin-controlled ratio:

```
maxVenueSharesIn = venueAssetsOut * maxSharesInRatio / 1e18
```

An untrusted caller could force the ALM to consume some or all of the difference between this cap and the fair shares required for `venueAssetsOut`. If the venue could be moved into an adverse withdrawal state, the withdrawal would succeed as long as the inflated share burn remained below the cap.

Examples of issues in generic ERC-4626 venues include:

1. **Temporary share-price manipulation:** Flash loans or manipulable `totalAssets()` computations could temporarily reduce assets per share.
2. **Deposit/withdraw fees:** Withdrawal (or deposit) fees could be triggered repeatedly and trigger an effective loss for the ALM funds each time. Note that these fees are usually percentage-based while the permissionless withdrawal contract only charges a fixed fee per withdrawal.
3. **Per-withdrawal rounding:** Rounding could charge a fixed extra share amount on each withdrawal. However, that should be mitigated by charging a **fixed fee** which would make the attack expensive.

Any excess fair value would be lost by the ALM while only `venueAssetsOut` of SparkVault liability would be extinguished. A searcher could capture it by owning remaining venue shares or sandwiching the internal unwind; otherwise the path could permit paid griefing. The fixed `Permissionless Withdrawals` penalty would not cap the loss because it is constant while the venue withdrawal scales.

No applicable mechanism was found for the currently used MetaMorpho V1.1 vaults used by the Spark ALM.

Recommendation: Keep `maxSharesInRatio` close to the venue's fair shares-per-asset ratio and treat any headroom as a venue-specific loss limit. Charge a high-enough penalty that discourages the attacks. When onboarding a new ERC-4626 venue, verify that none of the manipulation or fee scenarios described above apply.

Sky: Addressed in [PR 4](#).

Cantina Managed: Fix verified.

3.1.2 Protect user withdrawals against front-run penalty increases

Severity: Low Risk.

Context: `PermissionlessWithdrawals.sol#L237`.

Description: A withdrawal can be front-run by a penalty increase, causing the user to receive fewer assets than expected.

Recommendation: Add a user-provided `minAssets` parameter and revert when the withdrawal output is below this amount.

Sky: Addressed in [PR 4](#).

Cantina Managed: Fix verified.

3.1.3 Malicious venue can block itself from being removed via `setVenueType`

Severity: Low Risk.

Context: `PermissionlessWithdrawals.sol#L143`.



Description: The `setVenueType` function can be used to delist a venue again by setting `venueType` to `None`. However, that can revert during the external `asset()` call, allowing a malicious or broken venue to prevent itself from being disabled.

Recommendation: Handle the `None` path separately and perform asset validation only when configuring an active venue type:

```
// For venueType == None, avoid the external asset() call because it may
// be malicious and cause a denial of service when disabling the venue.
if (venueType != VenueType.None) {
    address asset = IERC4626Like(vault).asset();

    require(
        (venueType == VenueType.AAVE &&
         IATokenLike(venue).UNDERLYING_ASSET_ADDRESS() == asset) ||
        //...
    );
}
```

Sky: Addressed in PR 4.

Cantina Managed: Fixed by relying on `&&` and `||` lazy eval.

3.1.4 The penalty is charged when no ALM liquidity is used

Severity: Low Risk.

Context: `PermissionlessWithdrawals.sol#L263-L308`.

Description: The fixed penalty is always deducted from the redeemed amount. This also happens when the vault already has enough idle liquidity and neither the venue nor the ALMProxy provides any assets.

The technical documentation describes the penalty as compensation for force-unwinding ALM liquidity, but the user pays it even when no unwind or proxy transfer occurs.

Recommendation: Clarify that the penalty is an unconditional fee for using the backup path. If it is intended to cover ALM activity, consider charging it only when `assetsToTransfer > 0` or `assetsToWithdraw > 0`.

Sky: Acknowledged. We have decided to not expose such a view function and will keep the flow/logic the same, charging a fee for any withdrawals, even if no unwinding is needed.

Cantina Managed: Acknowledged.

3.2 Informational

3.2.1 Minor issues, Typos & Documentation

Severity: Informational.

Context: (See each case below).

Description:

- 1. `PermissionlessWithdrawals.sol#L405`: The internal `token` parameter refers to the `venue` (4626 vault) but is ambiguous and may be confused with the `venue`'s `asset` instead. Consider renaming `token` to `venue` for consistency with the contract's terminology.
- 2. `IPermissionlessWithdrawals.sol#L25`: The `NONE` enum value is used to indicate that a venue is not whitelisted which is unintuitive. Consider renaming `NONE` to a clearer value such as `UNSET` or `DISABLED`.
- 3. `IPermissionlessWithdrawals.sol#L160`: The documented exchange-rate scaling omits the asset-to-share decimal ratio. For a Morpho USDC vault with 6 asset decimals and 18 share decimals, a 1:1 rate must be represented as $1e18 * (1e18 / 1e6) = 1e30$, not `1e18`.



- 4. `PermissionlessWithdrawals.sol#L381`: The configured PSM `venue` address is used only as an identifier. It does not determine the PSM contract that the controller will actually call. Any configured address can be used as the PSM venue identifier. Clarify if this is intended.
- 5. `IPermissionlessWithdrawals.sol#L251`: The documented return value is incomplete because the function may also return the DPAU controller address, depending on the controller used.
- 6. `IPermissionlessWithdrawals.sol#L121` The `PermissionlessWithdraw` event does not identify the `venue` associated with the withdrawal, limiting traceability and off-chain monitoring.
- 7. `PermissionlessWithdrawalsLegacyPAU.sol#L28` and `PermissionlessWithdrawalsDiamondPAU.sol#L28`: `_USDS_CONVERSION_PRECISION` is defined with the same value in both implementations. Consider moving it to the abstract `PermissionlessWithdrawals` contract to avoid duplication.
- 8. `PermissionlessWithdrawals.sol#L259`: Liquidity is sourced using `convertToAssets(shares)`, while the final payout uses the amount returned by `redeem`. These values match for the current SparkVault implementation but are not guaranteed to match for arbitrary ERC-4626 vaults, particularly fee-bearing vaults. Consider documenting that only SparkVault implementations are supported, or explicitly handling the difference.
- 9. `PermissionlessWithdrawals.sol#L153`: The third-party `venue`'s `asset()` is only checked to match the spark vault's asset once during setup, not again during `permissionlessWithdraw`. A malicious or broken vault might have changed afterwards. However, not receiving the expected token should revert during the transfer, so there is no security risk. Consider documenting this.
- 10. `PermissionlessWithdrawalsDiamondPAU.sol#L6`: The dpau controller's interface depends on how its facets are wired from the Beacon. The assumed wiring is based on `ExternalFacetInterfaces.sol` and the current live Beacon state. Consider documenting this assumption.
- 11. `PermissionlessWithdrawals.sol#L264-L281`: Controller withdrawal and transfer hooks are only invoked when the vault lacks sufficient liquidity. Consequently, disabling venue operations or their rate limits does not fully pause permissionless withdrawals: withdrawals can continue while the vault holds enough assets to satisfy them directly. Consider documenting the emergency-pause procedure, including that governance must de whitelist the vault or revoke the controller's role on the ALMProxy to stop all permissionless withdrawals.

Recommendation: Consider addressing the issues mentioned above.

Sky: Addressed in [PR 4](#) and [PR 7](#).

Cantina Managed: All highlighted points were addressed except points 1.

3.2.2 Permissionless withdrawals share rate-limit capacity with other allocators

Severity: Informational.

Context: [TECH_DOC.md#L61](#).

Description: `PermissionlessWithdrawals` calls the same PAU functions and consumes the same rate-limit keys as other `ALLOCATOR_ROLE` holders. The limits are keyed by the operation and venue, not by the allocator calling it.

This does not appear to support a separate permissionless allowance. For example, if the overall venue withdrawal limit is \$10 million but permissionless withdrawals should be limited to \$5 million, it is unclear how this would be configured without also reducing the capacity available to other allocators.

Recommendation: Clarify whether permissionless withdrawals are expected to share the full PAU allowance and how you would protect the PAU against a possible malicious `PermissionlessWithdrawals`.

Sky: Acknowledged. We're going to allow them to comingle, if it becomes an issue in practice we can always just offboard the module from the system. But the idea is that the penalty will prevent people from using/spamming while the SLL offchain automation is live, and if it goes down and people need to withdraw, only then will it make viable sense to use this.

Cantina Managed: Acknowledged.



3.2.3 Add a permit-based withdrawal entry point

Severity: Informational.

Context: `IPermissionlessWithdrawals.sol#L232-L244`.

Description: Users must approve the helper before calling `permissionlessWithdraw`, which requires a separate transaction but the Spark Vaults are supporting EIP-2612 permits. Consider adding a permit-based entry point:

```
function permissionlessWithdrawWithPermit(
    address vault,
    address venue,
    address recipient,
    uint256 shares,
    uint256 deadline,
    uint8 v,
    bytes32 r,
    bytes32 s
) external nonReentrant {
    IPermitLike(vault).permit(
        msg.sender,
        address(this),
        shares,
        deadline,
        v,
        r,
        s
    );
    _permissionlessWithdraw(vault, venue, recipient, shares, msg.sender);
}
```

The existing withdrawal logic could be moved into a shared internal function to avoid duplicating it between the permit and non-permit entry points.

Recommendation: Consider adding `permissionlessWithdrawWithPermit` for supported vaults and refactoring the common withdrawal flow into an internal function.

Sky: Acknowledged. Won't add in order to reduce complexity/scope. Its not needed, this won't be a contract thats used enough to necessitate having good UX.

Cantina Managed: Acknowledged.

3.2.4 Delisting a vault leaves its venue configuration active in storage

Severity: Informational.

Context: `PermissionlessWithdrawals.sol#L115-L130`.

Description: Calling `setVaultConfig(vault, penaltyAmount, false)` only updates the vault's penalty and whitelist status. Its `(vault, venue)` mappings remain unchanged, and venue-level `maxSharesInRatio` values also remain in storage.

Rewhitelisting the vault can therefore reactivate its previous venue configuration. This may be unexpected if delisting is treated as removing the integration, especially after a venue or vault upgrade.

Recommendation: Consider clearing the associated venue configuration when retiring a vault, adding an explicit cleanup function, or documenting that delisting only pauses the vault and that old routes must be removed separately.

Sky: Documented in PR 4.

Cantina Managed: Fix verified. `TECH_DOC` now states that explicitly ("Vault Whitelisting and Configuration Persistence"): disable ≠ clear routes; re-enable reuses old venue config unless admin unsets them separately.



3.2.5 Use `AdministeredAgent` for `PermissionlessWithdrawals` allocator permissions

Severity: Informational.

Context: [README.md#architecture](#).

Description: Consider assigning `ALLOCATOR_ROLE` to an `AdministeredAgent` and having the withdrawal contract execute through it as an actor. This would make withdrawal permissions easier to revoke during an emergency.

Recommendation: Evaluate this permission model and update the code and documentation to support the `AdministeredAgent` interaction.

Sky: Fixed in [PR 4](#) and [PR 7](#).

Cantina Managed: Fix verified.